

MACHINA PARTY

プログラム説明資料

ジャンル：マルチプレイ 3D アクション / パーティーゲーム

開発環境：C# / Unity6 | 開発期間：2026/4 ~ 制作中（2人制作）



横浜デジタルアーツ専門学校 ゲーム科3年 石川原 吉美

担当：ゲーム内容全般（GameManager・Tank関連システム・レースシステム等のプログラム実装）

C#

Unity6

NavMesh

CharacterController

Rigidbody 物理

Input System

※ タイトル画面イラストは AI 画像生成（ChatGPT）で制作したものを使用

開発体制・使用ツールについて

01 所要時間の概算

- 開発開始から現在まで約19週間（2026年4月～）
- 週3～4回 × 2～3時間ベースで継続中
- 自身の実装時間目安 約150時間（制作中のため今後増加）

02 開発体制・担当範囲

- 2人体制（自身 + 1名）
- 自身の担当：ゲーム内容全般（GameManager・Tank関連システム・レースシステム等のプログラム実装）

03 学校のFW・AI活用範囲

- 学校指定の共通フレームワークは不使用、ゼロから構築
- 実装のエラー・不明点はAIに質問し参考にした
- タイトル画面イラストはAI画像生成（ChatGPT）で制作
- ゲームロジックの設計・実装は自身で実施

04 外部アセット・ミドルウェア

- タイトル画面イラスト以外のグラフィックはフリー素材を使用し、Photoshopで加工
- ミドルウェア・外部SDKの組み込みはなし

アピールポイント

01 GameManager 設計

- ラウンド制御・生存判定・勝者判定を1クラスに集約。
- PlayerDataでCPU/Player・カラー・操作方法をメニューからゲームへ受け渡す。

02 TankMovement（移動）

- CharacterControllerで3D移動を実装。
- ゲームパッドは右スティックでカメラ基準の向きへ旋回補助（移動方向自体は不変）。
- 爆風ノックバックも加速度減衰で実装。

03 TankShooting（射撃）

- 長押しでチャージ。プレイヤーはボタンを離れた瞬間に発射、CPUのみ最大チャージで自動発射。
- 放物線計算でAIの着弾予測地点を算出。発射後はクールタイムをゲージ表示。

04 TankAI（CPU制御）

- NavMeshで最短経路を計算し敵を追跡（Seek）。近づきすぎると逃走（Flee）に遷移。
- 射程・壁チェックで射撃タイミングを自律判定。

05 TankHealth（HP管理）

- HPバーをLerpで滑らかにアニメーション。被弾後の無敵時間（点滅演出）。
- HP0以下でOnDeath()。爆発演出のあとタンクを非表示にする。

06 レースシステム

- CarMovementはマリカー式の速度制御でドリフト・ミニターボを実装。
- LapCounterで周回管理、人数に応じ分割画面カメラを自動生成。

アーキテクチャ全体図

Main Menu → TankMode Scene → RaceMode Scene の3シーン構成



PlayerData / CarManager.Setup() — メニューのCPU/Human・カラー・操作設定をScene間でGameManager／各Carへ受け渡し | **TankManager・CarManager** — それぞれSetup・Reset・操作許可禁止を管理

GameManager 設計

ゲーム進行管理 — ラウンド制御・生存判定・タンク生成

01 ラウンド制御

- GameLoop()がRoundStarting→RoundPlaying→RoundEndingを順に処理。
- m_Wins が m_NumRoundsToWin に達したらメインメニュー（シーン0）へ遷移し終了。

02 生存判定

- OneTankLeft()でactiveSelfなタンク数を毎フレーム計測。
- 1体以下になったらRoundEndingへ進む。

03 PlayerData

- IsComputer・TankColor・UsedPrefab・ControllIndexを保持する内部クラス。
- メニューからプレイヤー数分の配列でまとめて受け渡す。

04 SpawnAllTanks()

- 人数分Instantiateし、全生成後にSetup()で各TankManagerを初期化。
- 続けてSetCameraTargets()でカメラ追従対象を設定。

GameManager.cs 抜粋

```
// ラウンド進行のメインループ
private IEnumerator GameLoop()
{
    yield return StartCoroutine(RoundStarting());
    yield return StartCoroutine(RoundPlaying());
    yield return StartCoroutine(RoundEnding());

    if (m_GameWinner != null)
        SceneManager.LoadScene(0);
    else
        StartCoroutine(GameLoop());
}

// 生存者が1人以下か判定
private bool OneTankLeft()
{
    int numTanksLeft = 0;
    for (int i = 0; i < m_PlayerCount; i++)
        if (m_SpawnPoints[i].m_Instance.activeSelf)
            numTanksLeft++;

    return numTanksLeft <= 1;
}
```

GameManager — ゲーム進行フロー図

MainMenu → GameScene (ラウンドループ)

設定フロー



ラウンドループ (5ラウンド先取の場合)



TankMovement — プレイヤー移動

操作方式（3層構造）・爆風ノックバック・エンジン音

01 CharacterController移動

- Rigidbodyを使わずCharacterControllerで3D移動を実装。
- stepOffset・slopeLimit・skinWidthで段差や坂を自然に処理。

02 操作方式は3層構造

- 基本はtransform.forward基準のタンク型移動+Turn()で旋回。
- Gamepad右スティックは向き補助のみ（移動方向は不変）。完全カメラ相対のTPS移動はm_IsDirectControl=true時のみ。

03 爆風ノックバック

- AddExplosionForce()が距離減衰込みでm_ExplosionVelocityを計算。
- Lerpで徐々に減衰、無敵中(IsInvincible)は完全無効化。

04 エンジン音制御

- 移動入力の有無でm_EngineIdling/m_EngineDrivingを切替再生。
- 毎回ランダムなPitchを加えて自然な音に。

操作ロジックの切り分け（Move()内）

① **基本移動（常時）** transform.forward × m_MovementInputValue → Turn()で左右旋回

② **Gamepad右スティック** しきい値(m_LookDeadzone)超で RotateTowards のみ実行。移動量は①のまま

③ **m_IsDirectControl=true** camForward×move + camRight×turn を合成した完全カメラ相対移動

TankMovement.cs — Move() 抜粋

```
if (m_UsingGamepadScheme)
{
    if (m_GamepadLookActive)
        transform.rotation = Quaternion.RotateTowards(
            transform.rotation, lookRot, m_LookTurnSpeed*dt);
    moveDirection = transform.forward * m_MovementInputValue;
}
else if (m_IsDirectControl)
{
    // ここだけが真のカメラ相対移動（独立フラグ）
    moveDirection = camForward*m_MovementInputValue
        + camRight*m_TurnInputValue;
}
```

TankShooting — 射撃システム

チャージ発射・放物線着弾計算・エイム演出・クールタイム

01 発射：人間は手動、CPUは自動

- ボタン押下でm_CurrentLaunchForceがmin→maxへ増加（チャージ）。
- プレイヤーはfireReleasedで初めてFire()。フルチャージでも自動発射しない。CPUのみ満タン到達で即Fire()。

02 着弾位置の計算

- GetProjectilePosition()が初速と重力から二次方程式で着弾XZ座標を算出。
- CPUの射程判定・正面判定(dot>0.99)に使用。人間側には着弾表示はない。

03 エイム矢印の演出

- UpdateAimArrowVisual()を毎フレーム実行。クールダウン中は指定色、フルチャージ中は振動して発射待ちを演出。
- 矢印はプレイヤーカラーに一番近い専用スプライトを自動選択（無ければ色タイントにフォールバック）。

04 発射後のクールタイム

- m_ShotCooldownTimer分だけ再発射不可。プレイヤー・CPU両方のUpdateで毎フレーム減算。
- GetCooldownSliderValue()がゲージを発射時の値から空へLerpで滑らかに戻す。

AimSliderの動き（チャージ量 → 発射力）



- プレイヤー操作 → ボタンを離れた瞬間に発射（自動発射なし）
- CPU操作 → チャージがMAXに達した瞬間に自動発射

TankShooting.cs — Human vs CPU

```
// HumanUpdate() 抜粋
if (fireReleased)
{
    Fire(); // フルチャージでも自動発射しない
    return;
}

// ComputerUpdate() 抜粋
if (m_CurrentLaunchForce >= m_MaxLaunchForce && !m_Fired)
{
    m_CurrentLaunchForce = m_MaxLaunchForce;
    Fire(); // CPUは満タンで自動発射
}
```


TankAI — CPU制御

NavMesh追跡 (Seek) と逃走 (Flee) の2状態AI

01 ターゲット選定

- 0.3~0.6秒ごと (タンクごとにランダム化し負荷分散) に NavMesh.CalculatePath() で全タンクへの経路長を比較。
- 最短経路の相手を新たなターゲットとして選ぶ。

02 Seek : 追跡と停止照準

- 射程内かつ NavMesh.Raycast で壁が無ければ移動を止めて StartCharging()。
- 正面判定(dot>0.99)と着弾距離の両方が揃うまでチャージを保持し続ける。

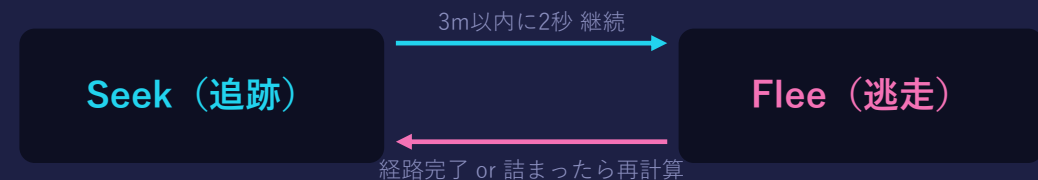
03 発射とクールタイム

- 条件成立で StopCharging() により発射し、m_TimeBetweenShot(2秒)のクールタイムへ。
- 相手が2秒以上静止していたら、発射直後にそのまま Flee(逃走)へ移行。

04 Flee : 近すぎる／詰まったら逃走

- 相手との距離が3.0m未満の状態が2秒続くと逃走開始。
- 相手と逆方向へ±90~180° ランダムに振った5~20m先へ経路生成。2秒以上動けなければ再計算。

状態遷移 (Seek ⇄ Flee)



TankAI.cs — SeekUpdate() 発射判定 抜粋

```
if (currentShotDistance >= targetDistance - 2 &&
    dotToTarget > 0.99f)
{
    m_IsMoving = false;
    m_Shooting.StopCharging();
    m_ShotCooldown = m_TimeBetweenShot;

    if (m_TimeSinceLastTargetMove > 2.0f)
        StartFleeing();
}
```

TankHealth — HP管理

ダメージ処理・被弾後無敵・死亡演出・HPバー演出

01 ダメージ処理 (TakeDamage)

- 無敵中(m_IsInvincible)または死亡済み(m_Dead)ならダメージを完全に無効化。
- HPをamount分だけ減算し、0未満にはしない。0以下になった瞬間にOnDeath()へ。

02 被弾後の無敵時間

- ダメージが通るたびInvincibleRoutine()を再起動 (Coroutine)。連続被弾でも時間はリセット。
- m_InvincibleTime秒の間、m_BlinkInterval間隔でRendererをON/OFFして点滅表示。

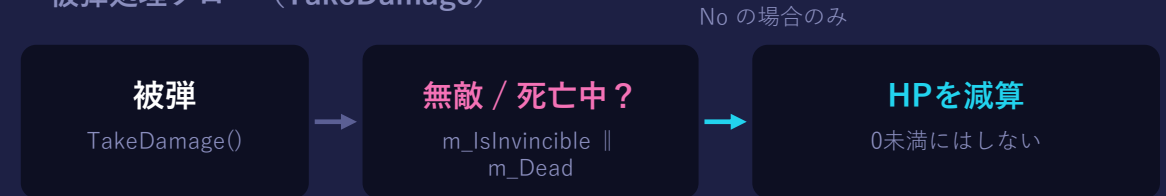
03 死亡処理 (OnDeath)

- m_Deadフラグで二重実行を防止。爆発パーティクル・爆発音を再生し、タンクをSetActive(false)で非表示に。
- 非表示になったタンクはGameManagerのOneTankLeft()集計から外れ、生存者数に反映される。

04 HPバー演出

- SetHealthUI()が毎フレームm_DisplayHealthをMathf.Lerpで実HPへ近づけてからSliderへ反映。
- 差が0.01未満ならスナップして誤差を防止。FillImageの色もColor.Lerpで緑⇄赤へ滑らかに変化。

被弾処理フロー (TakeDamage)



HP ≤ 0 : **OnDeath()** (非表示 + 爆発エフェクト)

HP > 0 : **無敵Coroutineを再始動** (点滅表示へ)

TankHealth.cs — TakeDamage() 抜粋

```
public void TakeDamage(float amount)
{
    if (m_IsInvincible || m_Dead) return;

    m_CurrentHealth = Mathf.Max(
        m_CurrentHealth - amount * (1f - m_ShieldValue), 0f);

    if (m_CurrentHealth <= 0f) { OnDeath(); return; }

    if (m_InvincibleCoroutine != null)
        StopCoroutine(m_InvincibleCoroutine);
    m_InvincibleCoroutine = StartCoroutine(InvincibleRoutine());
}
```

CarMovement — カート物理（マリカー式）

【新規追加】 速度スカラー直接制御・速度帯で変わる旋回・3段階ドリフト

01 速度スカラー直接制御

- currentSpeed(float)を指数Lerpで目標速度へ滑らかに追従させる。
- 毎FixedUpdateでforward × currentSpeedをrb.linearVelocityへ直接代入。

02 速度帯で変わる旋回レート

- forwardSpeedに応じ低速×0.5／中速×1.3／高速×1.75の3段階で変化。
- 停止付近は旋回不可。後退中はハンドルが反転する。

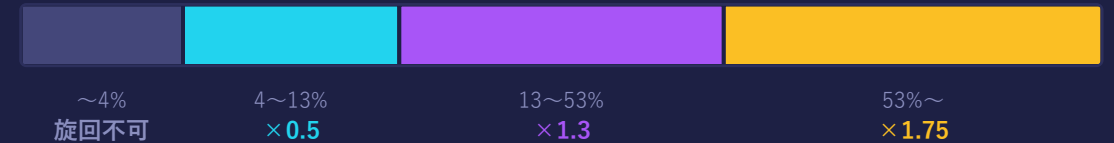
03 ドリフト & 3段階ミニターボ

- 押した瞬間のスティック方向で旋回方向をロック。反対に倒しても深さが変わるだけ。
- ゲージが60/120/240で段階アップ。放した瞬間、段階に応じ×1.25/1.5/1.75倍のブーストへ。

04 坂道姿勢とダート減速

- 接地Rayの地面法線にQuaternion.Slerpで車体を追従。80° 超は壁とみなす。
- Groundタグの地面は速度半減(ブースト中は無視)。downForceで飛び跳ねも抑制。

速度帯 → 旋回レート (forwardSpeed / maxSpeed)



● ● ● CarMovement.cs — FixedUpdate() 抜粋

```
// マリカー式：速度スカラーを直接代入（心臓部）
Vector3 vel = transform.forward * currentSpeed;

if (drifting)
{
    vel += -transform.right * (driftDirection *
        Mathf.Abs(currentSpeed) *
        driftSidewaysFriction * driftSteerRamp);
}

rb.linearVelocity = vel;
```

レースシステム — 周回・分割画面

チェックポイント順序判定・ラップ加算・最大4分割カメラ

01 チェックポイント順序判定

- CheckPoint通過時にLapCounter.PassCheckPoint(番号)を呼ぶ。
- currentCheckPointと一致した時だけ進む。順序違反や逆走は無視。

02 ゴール判定とラップ加算

- GoalTrigger通過時にLapCounter.PassGoal()を呼ぶ。
- 全チェックポイント通過済み(currentCheckPoint==1)の時だけcurrentLapを+1。

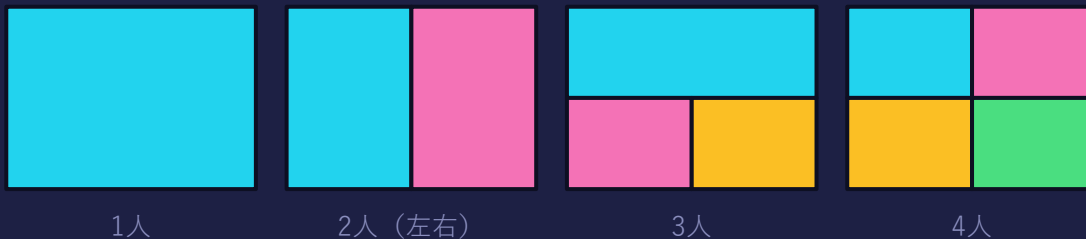
03 初周フラグ (hasCompletedFirstLap)

- Z座標がfirstLapEnableZ以上になるまでhasCompletedFirstLapはfalse。
- false中はチェックポイント・ゴール判定の両方を無効化し誤検出を防止。

04 分割画面カメラ

- RaceSplitScreenCameraManager.Setup()が人数に応じカメラ数を自動決定。
- 2人時はm_TwoPlayerLayoutで左右／上下分割を選択可能（最大4分割）。

分割画面レイアウト (GetViewports)



LapCounter.cs — PassGoal() 抜粋

```
public void PassGoal()
{
    if (!hasCompletedFirstLap) return;
    if (currentCheckPoint != 1) return; // 全CP未通過
    if (hasPassedGoalThisLap) return;   // 二重加算防止

    hasPassedGoalThisLap = true;
    currentLap++;
    ...
}
```

現状のまとめ



GameManager

ラウンド制対戦の進行・勝敗判定（Coroutine制御のゲームループ）



TankMovement

CharacterControllerによる3層移動（通常／Gamepad補助／カメラ相対）



TankShooting / TankAI

プレイヤー手動・CPU自動のチャージ発射と、NavMeshによるSeek/Flee制御



TankHealth / ShellExplosion

被弾後無敵点減・死亡時の爆発演出、距離減衰つきの爆風ダメージ



CarMovement（新規追加）

速度スカラー直接制御によるマリカー式操作感、3段階ドリフトブースト



レースシステム

チェックポイント順序判定・周回カウント・最大4分割画面カメラ

※ 本資料は実装コード（26スクリプト）と照合のうえ、記述内容を更新しています。